Sprint 4 – Week 2 Progress Report

SWENG 894 Capstone

MicroMatch

GitHub Repository Link

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace that connects students with short-term, skill-based professional opportunities. The platform currently supports the full project lifecycle from account creation through project completion.

The system architecture continues to follow a layered full-stack design:

- **Frontend:** React-based SPA (Single Page Application)
- **Backend:** FastAPI REST API
- **Database:** PostgreSQL (relational persistence)
- **Authentication:** JWT-based stateless authentication
- **Authorization:** Role-Based Access Control (RBAC)

Primary user roles:

- Student
- Organization
- Administrator

1.2 Sprint 1 – 3 Recap

Sprint 1 delivered the Minimum Viable Marketplace (MVP Foundation), including user authentication and registration (JWT), role-based access control (Student / Organization), student and organization profile management, project posting, and basic project browsing and search.

Sprint 2 introduced the Application Workflow Layer, enabling students to apply to projects, track application status, and allowing organizations to accept or reject applications (US-1 through US-9, FR-1 through FR-9).

Sprint 3 completed the core system lifecycle by implementing deliverable submission (FR-10), deliverable review (FR-11), project completion (FR-12), the feedback system (FR-13), admin moderation (FR-14), and system-wide logging middleware (FR-15). At the conclusion of Sprint 3, all foundational functional requirements had been satisfied and the platform supported a full end-to-end micro-internship lifecycle.

1.3 Sprint 4 Roadmap Context

Sprint 4 focuses on transforming MicroMatch from a workflow-complete marketplace into an intelligent, engagement-driven platform. The sprint introduces four major feature areas that were identified during earlier backlog refinement:

Planned Sprint 4 enhancements:

- Notifications system – FR-16 / US-16
- Messaging system – FR-17 / US-17
- Dashboard Analytics – FR-18 / US-18
- Profile enhancements – FR-19 / US-19
- Smart Matching & Ranking Algorithm – SA-1

Week 1 of Sprint 4 delivered the complete notifications feature (FR-16). Week 2 now targets the Smart Matching & Ranking Algorithm (SA-1 / US-20), which is the most technically complex milestone of the sprint and represents the capstone's Significant Algorithm requirement.

1.4 Functional Requirements to User Stories Mapping

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4
FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6
FR-8: The system shall track application status.	US-8

FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10
FR-11: The system shall allow orgs to review deliverables.	US-11
FR-12: The system shall allow marking projects complete.	US-12
FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14
FR-15: The system shall log system actions.	US-15
FR-16: The system shall notify users of key events (application decisions, deliverable reviews, project completion).	US-16
FR-17: The system shall allow users to send/receive messages within a project.	US-17
FR-18: The system shall provide users with basic analytics (applications, completed projects).	US-18
FR-19: The system shall allow profiles to display skills, completed projects, and ratings.	US-19
SA-1: Smart Matching & Ranking Algorithm.	US-20

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint 4 – Week 2 Overview

2.1 Sprint Goal

The overarching goal of Sprint 4 is:

- To transform MicroMatch from a workflow-complete marketplace into an intelligent engagement-driven platform through notifications, smart recommendations, messaging, analytics, and profile enhancements.

The goal of Sprint 4 – Week 2 is:

- To implement the Smart Matching & Ranking Algorithm (SA-1 / US-20), delivering an intelligent recommendation engine that scores and ranks open projects for students based on five weighted factors: skill alignment, experience fit, interest overlap, platform activity, and historical success.

3. Sprint Planning

3.1 Product Backlog Grooming and Reprioritization

At the start of Sprint 4, the team conducted a backlog refinement session to assess remaining enhancements deferred from Sprint 3. All Sprint 3 requirements (FR-10 through FR-15) were confirmed as completed. Estimation was performed using Planning Poker with Fibonacci values (3, 5, 8, 13). Stories were evaluated based on dependency relationships, impact on user engagement, risk of integration failure, and contribution to the capstone demonstration

Reprioritization Strategy

Notifications (FR-16) were designated as the Week 1 focus as the lowest-friction, highest-impact improvement that could be layered on top of existing workflow. The Smart Matching Algorithm (SA-1) was scheduled for Week 2 as the most technically complex milestone, benefiting from notifications already being in place. Messaging, analytics, and profile enhancements were deferred to Week 3.

3.2 Groomed Product Backlog

Priority Definitions

- Highest: Critical sprint milestone with significant algorithm or core infrastructure.

- High: Core MVP functionality required for end-to-end workflow.
- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting
US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction
US-7	Review Applications	Medium	5	Sprint 2	Depends on applications
US-8	Track Status	High	3	Sprint 2	Enhances UX
US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	3	Sprint 3	Completion validation
US-12	Mark Complete	Medium	3	Sprint 3	Project closure
US-13	Feedback	Low	5	Sprint 3	Post-completion feature
US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern
US-16	Notifications	Medium	5	Sprint 4	User awareness
US-17	Messaging	Medium	8	Sprint 4	Communication
US-18	Analytics	Low	5	Sprint 4	Insights

US-19	Profile Enhancements	Low	3	Sprint 4	UX Improvements
US-20 / SA-1	Smart Matching Algorithm	Highest	8	Sprint 4	Significant Algorithm

3.3 Sprint Backlog – Week 2 Commitment

Based on historical velocity and current capacity, the Week 2 Sprint 4 commitment is 18 story points across three GitHub issues that together deliver the complete notification feature end-to-end.

User Story	Description	Story Points
US-20/SA-1	Matching Score Engine	8
US-20/SA-1	Recommendation API Endpoint	5
US-20/SA-1	Recommendation UI Integration	5

Total Commitment: 18 Story Points

Capacity Planning and Velocity Justification

As a single-developer team, historical velocity across previous sprints averaged:

- Sprint 1: ~16 story points
- Sprint 2: ~14 story points
- Sprint 3: ~16 story points

This establishes a consistent velocity range of 14–16 story points per week. The Week 2 commitment of 18 story points is slightly above average velocity, justified by the fact that SA-1 is a focused, self-contained algorithmic feature. All three issues (#89, #90, #91) follow a clean backend → API → UI delivery path with no cross-cutting dependencies, enabling efficient sequential execution. The matching engine (Issue #89) itself is a pure Python service with no DB migration required, significantly reducing integration risk relative to features that require new ORM models.

Sprint 4 Week 2 Complexity Factors

Week 2 introduce:

- Weighted multi-factor scoring algorithm with five independent components
- Dataclass-based DTO layer decoupling the engine from SQLAlchemy entirely
- O(N log N) ranking over all open projects with top-N slicing support
- API layer assembling StudentDTO from six separate DB queries

- Frontend recommendation widget with per-card apply flow and score visualization
- 54 unit tests covering individual factors, formula integration, edge cases, and ranking

3.4 Sprint 4 Backlog Structure

Sprint 4 was divided into three weekly development phases.

Week 1 – Notification Workflow

Completed Issues

Issue	Description	Story Points	User Stories	Status
#76	Notification Event API	5	US-16	Done
#83	Notification Retrieval API	3	US-16	Done
#84	Notification UI Center	3	US-16	Done

Week 1 Total Commitment: 11 Story Points

Week 2 – Smart Matching Algorithm

Completed Issues

Issue	Description	Story Points	User Stories	Status
#89	Matching Score Engine	8	US-20	Done
#90	Recommendation API Endpoint	5	US-20	Done
#91	Recommendation UI Integration	5	US-20	Done

Week 2 Total Commitment: 18 Story Points

Week 3 — Messaging, Analytics, and Profile Enhancement Bundle

Planned Issues

Issue	Description	Story Points
#77	Project Messaging API	8
#85	Messaging UI Integration	5
#78	Analytics Summary API	5
#86	Analytics Dashboard UI	3
#79	Profile Enhancements API	3
#87	Profile UI Upgrade	3

Week 3 Total Commitment: 27 Story Points

4. UI Design

Wireframes were created for US-16 through US-20.

4.1 Notifications (US-16)

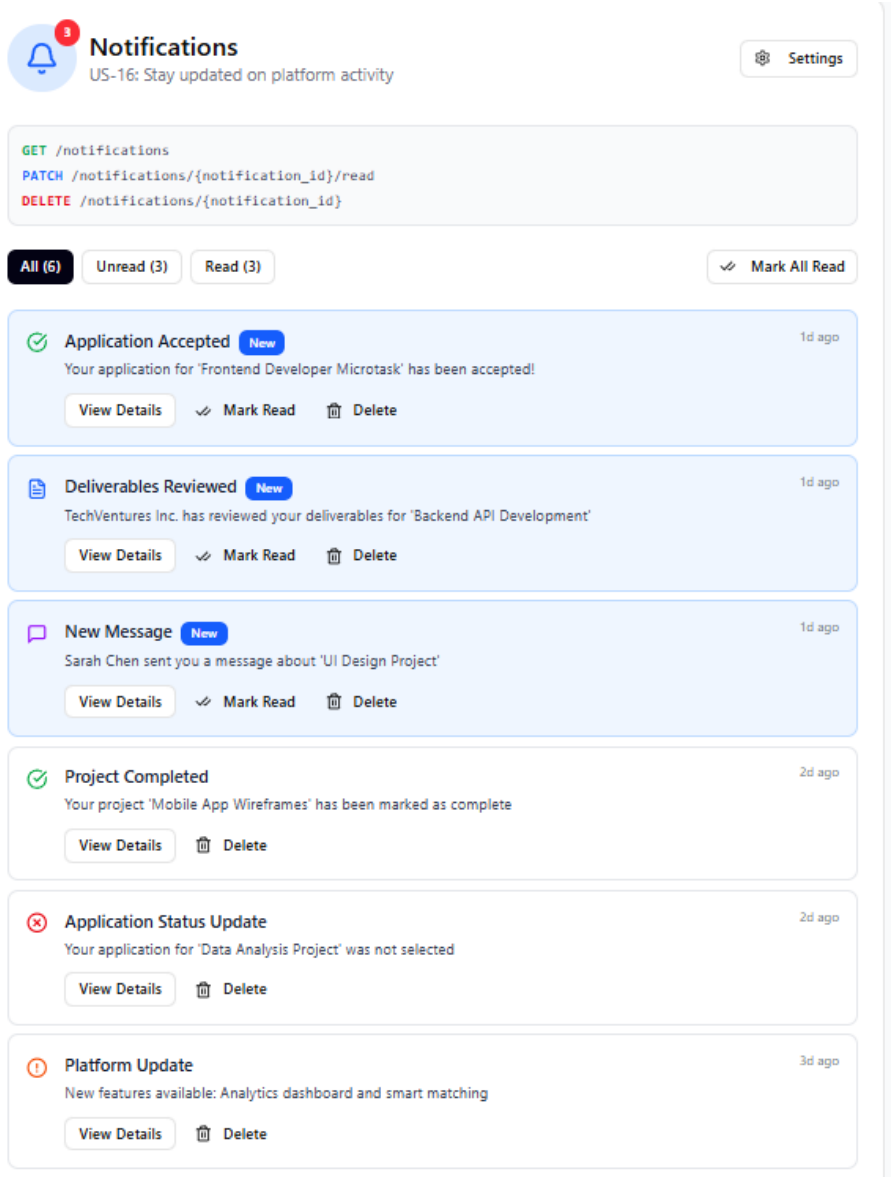


Figure 1 shows the wireframe for US-16 (Notifications Pages.

Description and Design Rationale

The Notifications page provides authenticated users with a centralized notification center to stay informed about critical platform events and activities. This page directly supports:

This interface enables users to view, filter, and manage notifications related to application statuses, deliverable reviews, project completions, messages, and system announcements. The notification system integrates with all major platform workflows (US-8, US-11, US-12, US-17) to provide timely awareness of important events.

Layout Structure

The interface is structured into three primary sections:

1. Header Section
    a. Bell icon with unread count badge
    b. Page title and user story reference
    c. Settings button for notification preferences
    d. API endpoint documentation
2. Filter Bar
    a. "All" notifications button with total count
    b. "Unread" filter with unread count
    c. "Read" filter with read count
    d. "Mark All as Read" action button (conditional)
3. Notifications List
    a. Notification cards with read/unread visual states
    b. Type-specific icons (CheckCircle, FileText, MessageSquare, etc.)
    c. Title and message content
    d. Relative timestamps (e.g., "5h ago", "2d ago")
    e. Priority indicators (high, medium, low)
    f. Action buttons (View Details, Mark Read, Delete)

Usability Considerations

Visual Hierarchy:

- Unread notifications displayed with blue background (bg-blue-50)
- Read notifications with white background
- "New" badge for unread items
- Color-coded icons by notification type

Status Badges:

- Unread count prominently displayed in red badge on bell icon
- Real-time count updates as notifications are read
- Visual separation between notification states

Timestamp Formatting:

- Relative time for recent notifications (minutes, hours)
- Absolute dates for older notifications
- Human-readable format reduces cognitive load

Action Accessibility:

- Individual "Mark Read" button for each notification
- Bulk "Mark All as Read" when unread notifications exist
- Delete action with confirmation (via toast)
- "View Details" navigation to relevant pages

Empty State Handling:

- Different messages based on filter context
- "All caught up!" for unread filter with no items
- "You have no notifications yet" for general empty state
- BellOff icon for visual clarity

Filtering Efficiency:

- Three-state filter (All, Unread, Read)
- Count indicators for each filter
- Active filter highlighted with default variant
- Instant filter application without page reload

Validation & Error Handling

Notification Loading:

- Graceful handling of empty notification list
- Loading state during initial fetch
- Error message if notifications fail to load

Action Validation:

- Mark as read only available for unread notifications
- Delete action confirms via toast message
- Prevents double-click on action buttons

State Synchronization:

- Real-time unread count updates
- Filter results update immediately
- Consistent state across browser tabs (future enhancement)

Network Error Handling:

- Retry mechanism for failed requests
- Offline notification queue (future enhancement)
- Error toast for failed actions

Empty State Conditions:

- No notifications exist: "You have no notifications yet"
- All read (unread filter): "All caught up!"
- Filter returns no results: Contextual empty message

Security Alignment

- **Accessible only to:** All authenticated users (students and organizations)
- JWT authentication required
- **User-specific filtering:** Only show notifications for authenticated user
- **Privacy protection:** No exposure of other users' notification data
- **Audit trail:** Notification read/delete actions logged (US-15)
- **XSS prevention:** Sanitized notification message content
- **Rate limiting:** Prevent notification spam

Alignment With Backend Design

This UI maps directly to:

Endpoints:

- GET /notifications
- PATCH /notifications/{notification_id}/read
- DELETE /notifications/{notification_id}

GET Response Schema Includes:

```
{
  "notifications": [
```

```
  {
    "id": "notif_001",
    "type": "application_accepted" | "deliverable_reviewed" | "new_message" | "project_completed" | "application_rejected" | "system_update",
    "title": "Notification title",
    "message": "Notification message content",
    "read": false,
    "createdAt": "2026-04-15T14:30:00Z",
    "actionUrl": "/student-dashboard",
    "priority": "high" | "medium" | "low",
    "icon": "CheckCircle" | "FileText" | "MessageSquare" | "XCircle" | "AlertCircle"
  }
],
"unreadCount": 3,
"pagination": {...}
}
```

PATCH Request (Mark as Read):

```
{
  "read": true
}
```

Notification Types and Triggers:

1. Application Status Changes (US-8):
    a. **Application accepted:** High priority
    b. **Application rejected:** Low priority
    c. **Triggers:** Organization accept/reject actions

2. Deliverable Events (US-11):
    a. **Deliverables reviewed:** High priority
    b. **Feedback provided:** Medium priority
    c. **Triggers:** Organization review actions
3. Project Completion (US-12):
    a. **Project marked complete:** Medium priority
    b. **Feedback request:** Medium priority
    c. **Triggers:** Organization completion action
4. Messaging (US-17):
    a. **New message received:** Medium/High priority
    b. **Urgent message flag:** High priority
    c. **Triggers:** Message sent by project partner
5. **System Notifications:**
    a. **Platform updates:** Low priority
    b. **Maintenance notices:** Medium priority
    c. **Policy changes:** Low priority
    d. **Triggers:** Admin announcements

State Transitions:

- Notification created: read: false
- User views notification: read: true
- User deletes notification: Soft delete or permanent removal

4.2 Messaging (US-17)

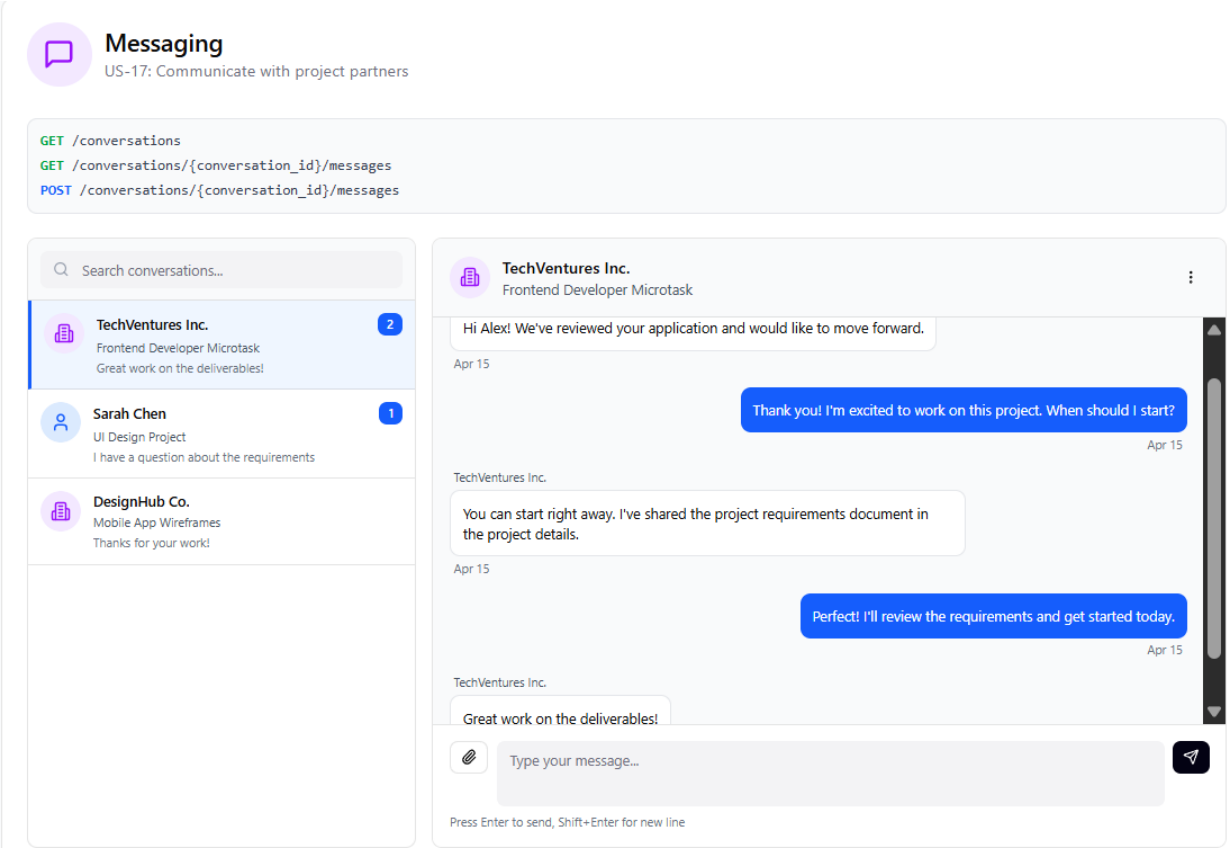


Figure 2 shows the wireframe for US-17 (Messaging).

Description and Design Rationale

The Messaging page enables authenticated users to communicate directly with project partners through secure, project-scoped conversation threads.

This interface provides students and organizations with a real-time messaging platform to discuss project requirements, clarify expectations, share progress updates, and maintain professional communication throughout the project lifecycle. All conversations are project-specific and accessible only to project participants.

Layout Structure

The interface is structured into two primary sections with a split-pane layout:

1. Header Section
    a. MessageSquare icon (purple theme)
    b. Page title and user story reference
    c. API endpoint documentation

2. Conversations List (Left Pane - 1/3 width)
    a. Search bar with icon
    b. Scrollable conversation list
    c. Each conversation card displays:
        i. Avatar with role indicator (Organization/Student)
        ii. Participant name
        iii. Project title
        iv. Last message preview
        v. Unread badge count
        vi. Active state highlighting
3. Message Thread (Right Pane - 2/3 width)
    a. Conversation header with:
        i. Participant avatar and name
        ii. Project title
        iii. Options menu button
    b. Message area (scrollable):
        i. Individual message bubbles
        ii. Sender name (for received messages)
        iii. Message content
        iv. Timestamp
        v. Alignment (own messages right, received left)
    c. Message input section:
        i. Attachment button
        ii. Multi-line textarea
        iii. Send button
        iv. Keyboard shortcut hint

Usability Considerations

Conversation Navigation:

- Search functionality across participant names and project titles
- Active conversation highlighted with blue left border and background
- Unread count badges prominently displayed
- Hover states for conversation cards
- Automatic scrolling to newest conversation

Message Display:

- **Own messages:** Right-aligned, blue background, white text

- **Received messages:** Left-aligned, white background, bordered
- Sender name displayed for received messages
- Relative timestamps for context
- Clear visual distinction between parties

Real-Time Interaction:

- Instant message delivery
- Typing indicators (future enhancement)
- Read receipts (future enhancement)
- Auto-scroll to latest message
- Message delivery confirmation

Keyboard Shortcuts:

- Enter to send message
- Shift+Enter for new line
- Visible hint text below input
- Prevents accidental sends

File Attachments:

- Paperclip button for file selection
- Support for common file types
- File preview in conversation
- Size limit enforcement

Empty State:

- Clear messaging when no conversation selected
- MessageSquare icon with instruction text
- Prompts user to select a conversation

Search Functionality:

- Real-time filtering of conversations
- Searches both participant name and project title
- Clear search results with no matches message

Validation & Error Handling

Message Requirements:

- Cannot send empty messages

- Client-side validation before submission
- Error toast: "Cannot send empty message"
- Prevents whitespace-only messages

Access Control:

- Only project participants can access conversation
- 403 error if unauthorized user attempts access
- Project relationship validation server-side

Character Limits:

- Message content: 0-5000 characters
- Graceful handling of length exceeded
- Character counter on long messages (future)

Network Error Handling:

- Retry mechanism for failed sends
- Message queue for offline mode
- Visual indicator for failed messages
- Resend option for failures

File Upload Validation:

- File type whitelist enforcement
- Maximum file size limit (10MB)
- Virus scanning before delivery
- Upload progress indicator

Empty Conversation State:

- "No conversations yet" message
- Prompts to start communicating with project partners

Security Alignment

- **Accessible to:** Students and organizations involved in the same project
- JWT authentication required
- **Project participation validation:** Server-side verification
- **Encryption:** Messages encrypted in transit (TLS)
- **Message retention:** Preserved for audit purposes (US-15)
- **Content moderation:** Ability to report inappropriate messages

- **Admin access:** Flagged conversations reviewed by admins (US-14)
- **No PII exposure:** Sensitive data exchange prohibited
- **Audit trail:** All messages logged with sender, timestamp, project context

Alignment With Backend Design

This UI maps directly to:

Endpoints:

- GET /conversations
- GET /conversations/{conversation_id}/messages
- POST /conversations/{conversation_id}/messages

Project-Scoped Conversations:

- One conversation per project
- Auto-created when student accepted (US-8)
- Participants: Student + Organization
- Conversation persists after project completion
- Archived conversations (future enhancement)

Message Triggers:

- New message notification sent (US-16)
- Email notification for unread messages (configurable)
- Browser push notification option
- Unread count updated real-time

Appropriate Use Cases:

- Project clarification questions
- Requirement discussions
- Progress updates
- Timeline coordination
- Professional communication

Prohibited Content:

- Personal contact information exchange
- Payment discussions outside platform
- Inappropriate or harassing content
- Spam or solicitation

- Confidential company data

Moderation System:

- Report message functionality
- Flagged messages reviewed by admins (US-14)
- Automated content filtering
- User blocking capability
- Conversation archiving

4.3 Analytics (US-18)

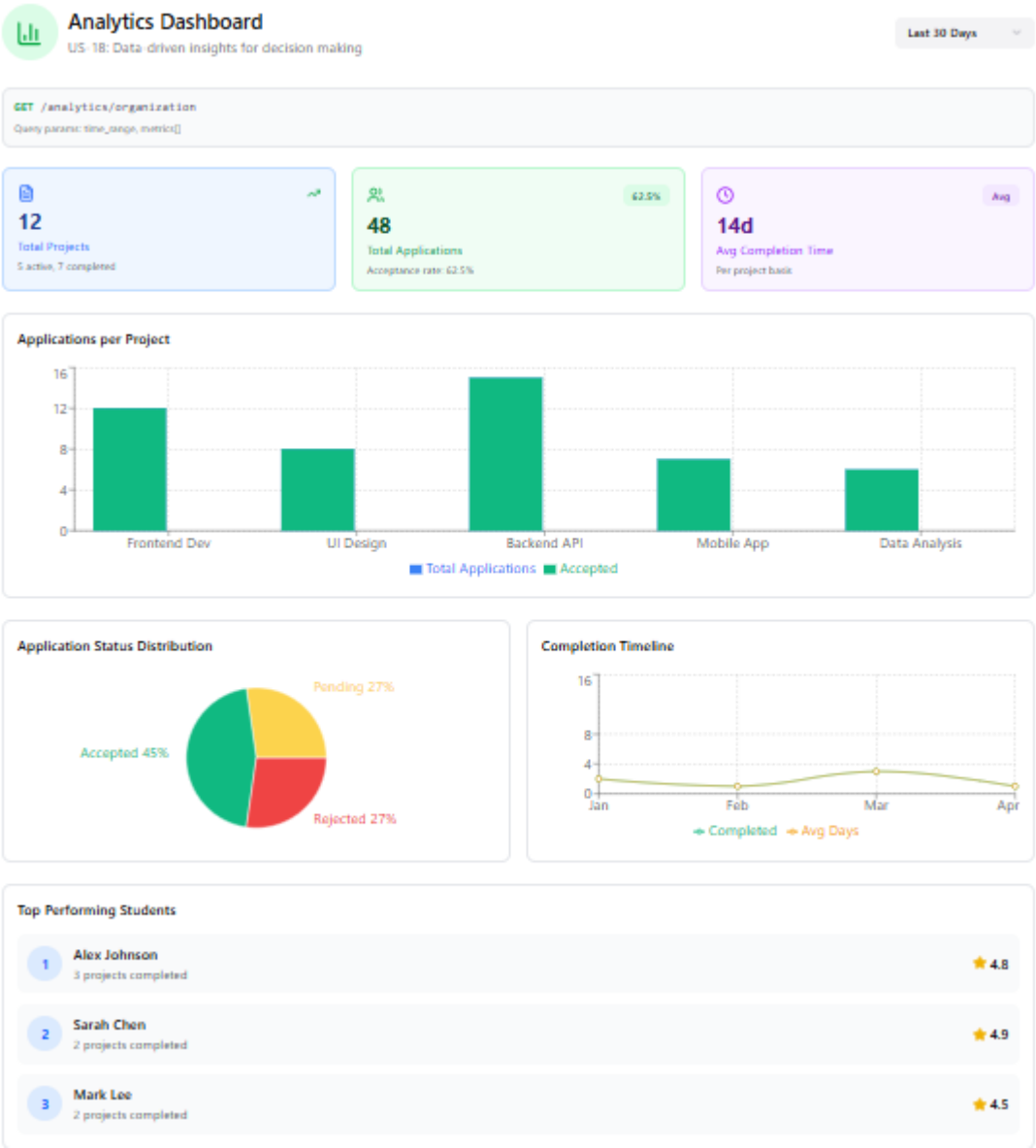


Figure 3 shows the wireframe for US-18 (Analytics).

Description and Design Rationale

This interface enables organizations to analyze project performance, application conversion rates, and student quality metrics, while students can track their application success rates, project completion statistics, and skill development. The analytics system provides actionable insights through visualizations, trend analysis, and comparative benchmarking.

Layout Structure

The interface is structured into role-specific sections:

Common Header Section:

1. BarChart3 icon (green theme)
2. Page title and user story reference
3. Time range selector dropdown (7 days, 30 days, 90 days, 1 year)
4. API endpoint documentation

Organization Analytics View:

1. Overview Statistics Cards (3 columns)
    a. Total Projects (with active/completed breakdown)
    b. Total Applications (with acceptance rate)
    c. Average Completion Time (in days)
    d. Each card has icon, metric, description, and detail
2. Applications per Project Chart
    a. Bar chart visualization
    b. Total applications vs. accepted per project
    c. Comparison across all projects
    d. Responsive container
3. Application Status Distribution & Completion Timeline (2 columns)
    a. Pie chart: Pending, Accepted, Rejected distribution
    b. Line chart: Completion trend over time
    c. Color-coded segments
    d. Top Performing Students List
4. Ranked display with position number
    a. Student name and project count
    b. Star rating display
    c. Sortable by performance

Student Analytics View:

1. Overview Statistics Cards (3 columns)
    a. Total Applications Submitted (with pending count)
    b. Projects Completed (with accepted applications)
    c. Average Rating (based on completed projects)
2. Application Trend Chart
    a. Line chart showing applied vs. accepted over time
    b. Month-over-month comparison
    c. Success rate visualization
3. Skills Portfolio
    a. Progress bars for each skill
    b. Proficiency percentage display
    c. Visual skill level indicators
    d. Growth tracking capability
4. Insights Information Panel
    a. Real-time metrics notice
    b. Export capabilities
    c. Customizable filters
5. Analytics Types Grid
    a. Organization-specific metrics (left)
    b. Student-specific metrics (right)

Usability Considerations

Time Range Filtering:

- Dropdown selector for quick filtering
- Pre-defined ranges (7 days, 30 days, 90 days, 1 year)
- Persistent selection across page refresh
- Real-time chart updates on selection

Color-Coded Statistics:

- Blue: Project/application metrics
- Green: Success and completion metrics
- Purple: Time-based analytics
- Yellow: Rating and quality metrics
- Consistent color scheme across all visualizations

Interactive Charts:

- Hover tooltips showing exact values
- Legend for multi-series charts
- Responsive scaling for different screen sizes
- Grid lines for value estimation
- Clear axis labels

Metric Comparisons:

- Side-by-side statistics for quick comparison
- Trend indicators (up/down arrows)
- Percentage calculations displayed
- Contextual detail text under main metrics

Data Visualization:

- Bar charts for categorical comparisons
- Pie charts for distribution analysis
- Line charts for temporal trends
- Progress bars for skill levels
- Appropriate chart types for data patterns

Empty State Handling:

- "No data available for this time period"
- Suggestions to adjust time range
- Encouragement to complete more projects

Validation & Error Handling

Data Loading:

- Loading state during data fetch
- Skeleton placeholders for charts
- Error message if data fails to load
- Retry mechanism for failed requests

Time Range Validation:

- Server-side validation of date parameters
- Maximum range limit to prevent performance issues
- Default to 30 days if invalid range provided

Access Control:

- Role-specific data exposure
- Students see only their own analytics
- Organizations see only their project analytics
- No cross-user data leakage

Metric Calculation Errors:

- Safe division handling (avoid divide by zero)
- Null/undefined data handling
- Default values for missing data
- Clear "N/A" indicators when data unavailable

Export Functionality:

- CSV/PDF export validation
- File size limits for large datasets
- Download timeout protection
- Error handling for failed exports

Security Alignment

- Accessible to: All authenticated users (role-based data)
- JWT authentication required
- Data isolation: Users only see their own analytics
- Aggregated data only: Individual user privacy protected
- No PII exposure: Anonymous aggregation for benchmarks
- Rate limiting: Prevent excessive analytics queries
- Audit logging: Analytics access tracked (US-15)

Alignment With Backend Design

This UI maps directly to:

Endpoint:

- GET /analytics/organization (for organizations)
- GET /analytics/student (for students)

Query Parameters:

?time_range=7days|30days|90days|1year
&metrics[]=projects,applications,ratings,completion_time

Metric Calculations:

- Organization Metrics:
  - Acceptance Rate: (Accepted Applications / Total Applications) × 100
  - Average Completion Time: Sum of project durations / Completed projects
  - Project Success Rate: (Completed projects / Total projects) × 100
  - Application Conversion: (Accepted / Total applications) per project
- Student Metrics:
  - Application Success Rate: (Accepted / Total applications) × 100
  - Average Rating: Sum of ratings / Number of rated projects
  - Skill Level: Based on project completions and feedback
  - Portfolio Growth: Projects completed over time

Data Aggregation:

- Real-time calculations from database
- Cached for performance (5-minute TTL)
- Incremental updates on new events
- Historical data preserved for trends

Export Formats:

- CSV: Tabular data export for spreadsheet analysis
- PDF: Formatted report with charts
- Date range included in export
- User-specific data only

Analytics Integration Points:

- Application events (US-8)
- Deliverable submissions (US-10)
- Project completions (US-12)
- Feedback ratings (US-13)
- Smart matching performance (US-20)

4.4 Profile Enhancements (US-19)

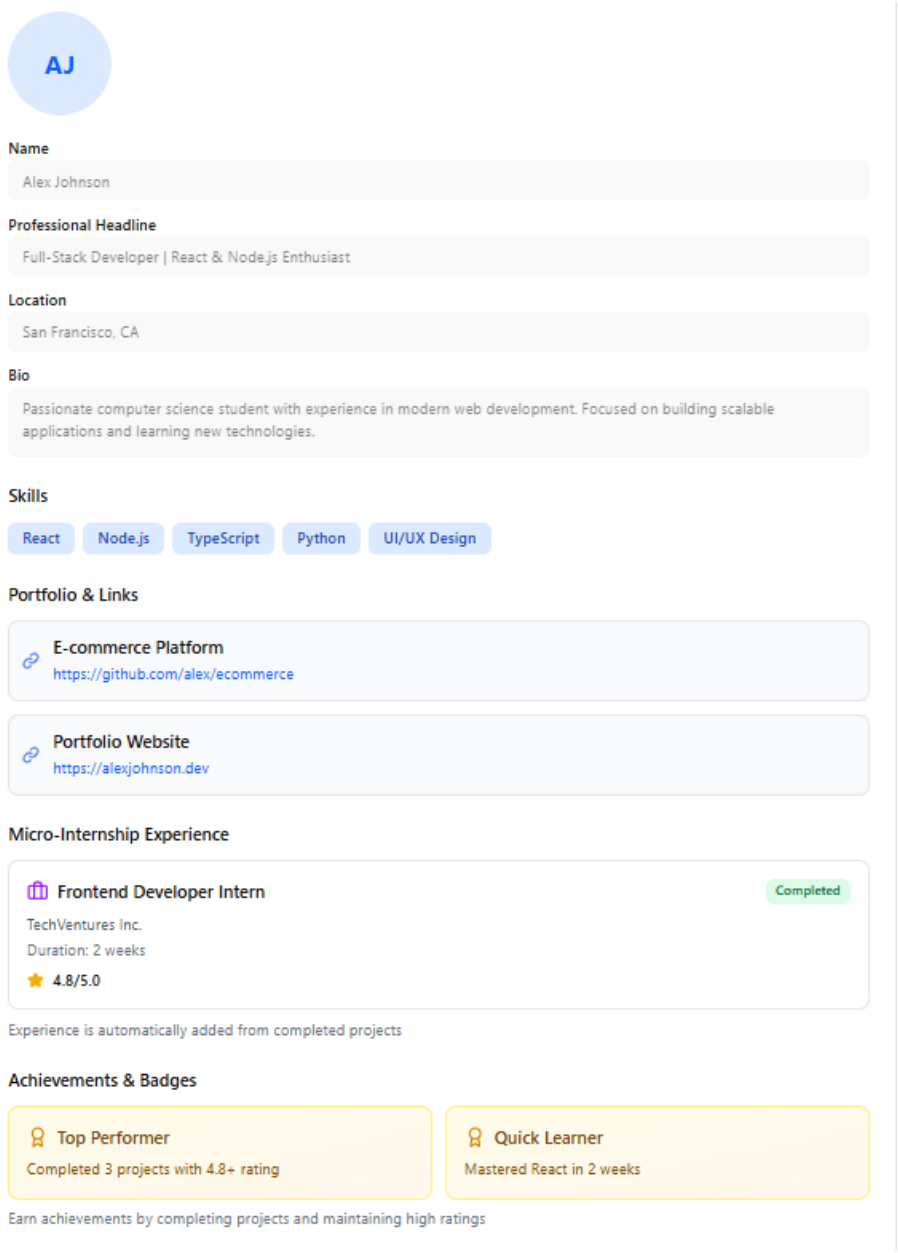


Figure 4 shows the wireframe for US-19 (Profile Enhancements).

Description and Design Rationale

The Profile Enhancements page enables authenticated users to build comprehensive professional profiles that showcase their skills, experience, achievements, and portfolio.

This interface provides students with the ability to create rich profiles that increase their visibility in the smart matching algorithm (US-20) and improve their application

acceptance rates. The enhanced profile includes professional headlines, skills tags, portfolio links, automatic experience tracking, achievement badges, and profile completeness indicators.

Layout Structure

The interface is structured into multiple sections with edit/view modes:

1. Header Section
    a. User icon (indigo theme)
    b. Page title and user story reference
    c. Mode-specific action buttons:
    d. View mode: Preview and Edit buttons
    e. Edit mode: Cancel and Save Changes buttons
    f. API endpoint documentation
2. Profile Picture Section
    a. Large avatar with initials fallback
    b. Upload Photo button (edit mode only)
    c. Prominent placement for visual identity
3. Basic Information Section
    a. Name input field
    b. Professional Headline input (e.g., "Full-Stack Developer | React Specialist")
    c. Location input field
    d. Bio textarea (multi-line)
    e. All fields disabled in view mode
4. Skills Section
    a. Visual skill tags (blue badges)
    b. Remove button per skill (edit mode, X icon)
    c. Add skill input with "Add" button
    d. Enter key support for quick adding
    e. Prevents duplicate skills
5. Portfolio & Links Section
    a. List of portfolio items with:
        i. Link icon
        ii. Project title
        iii. Clickable URL (opens in new tab)
        iv. Remove button (edit mode)
    b. Add portfolio item form (edit mode):
        i. Project title input

        ii. URL input
        iii. Type selector (GitHub, Website, etc.)
        iv. Add button
6. Micro-Internship Experience Section
    a. Auto-generated from completed projects
    b. Each experience card shows:
        i. Briefcase icon
        ii. Position title
        iii. Organization name
        iv. Duration
        v. Completion badge
        vi. Star rating
    c. Read-only (auto-populated)
7. Achievements & Badges Section
    a. Grid layout (2 columns)
    b. Each achievement card:
        i. Award icon
        ii. Achievement title
        iii. Description
        iv. Gradient background (yellow to orange)
    c. Auto-earned based on platform activity
8. Profile Enhancement Features Panel
    a. Feature list and benefits
    b. Integration with smart matching (US-20)
9. Profile Completeness Indicator
    a. Percentage badge
    b. Visual progress bar
    c. Checklist of remaining items to reach 100%
    d. Motivational messaging
10. Enhanced Profile Benefits Panel
    a. Visibility and matching improvements
    b. Application acceptance rate impact
    c. Professional presentation value

Usability Considerations

Edit Mode Toggle:

- Clear separation between view and edit modes

- Edit button prominently displayed
- Save/Cancel buttons in edit mode
- Prevent data loss with cancel confirmation

Profile Picture:

- Large avatar for visual identity
- Initials fallback when no photo uploaded
- Upload button with camera icon
- Image size and format validation

Skills Management:

- Visual tag-based display (easy scanning)
- Quick add with Enter key support
- Duplicate prevention
- Remove with single click in edit mode
- Alphabetical sorting (future enhancement)

Portfolio Organization:

- Clean card-based layout for each item
- External link icon for clarity
- Type badges (GitHub, Website, etc.)
- URL validation on add
- New tab opening for security

Experience Auto-Generation:

- Automatic population from completed projects (US-12)
- No manual entry required
- Includes ratings from feedback (US-13)
- Duration calculated automatically
- Organization name linked to project

Achievement Gamification:

- Visual badges with gradient backgrounds
- Clear criteria for earning

- Motivates profile completion and quality work
- Automatic awarding based on metrics

Profile Completeness:

- Real-time percentage calculation
- Visual progress bar
- Clear checklist of next steps
- Encourages 100% completion
- Higher completeness = better matching (US-20)

Form Validation:

- Required field indicators
- Character count for bio
- URL format validation
- Inline error messages
- Success toast on save

Validation & Error Handling

Field Validation:

- Name: Required, 2-100 characters
- Headline: Optional, max 150 characters
- Location: Optional, standard format
- Bio: Optional, max 1000 characters
- Skills: Non-empty, unique, max 50 skills
- Portfolio URL: Valid URL format

Duplicate Prevention:

- Skills: Cannot add existing skill
- Portfolio: Check for duplicate URLs
- Client-side validation before add

Save Validation:

- All required fields must be filled
- At least one skill required (recommended)

- Bio should be 50+ characters (recommended)
- Server-side validation on PATCH request

Error Handling:

- Network errors: Retry option with toast
- Validation errors: Inline field errors
- Save conflicts: Stale data detection
- Cancel warning if unsaved changes exist

Empty States:

- No skills: "Add your first skill to get started"
- No portfolio: "Showcase your projects and work"
- No achievements: "Earn achievements by completing projects"

Image Upload Errors:

- File too large: Max 5MB
- Invalid format: JPG, PNG only
- Upload failure: Retry option
- Malware detection: Rejection with message

Security Alignment

- Accessible to: Profile owner only
- JWT authentication required
- Ownership validation: Cannot edit other users' profiles
- Public view: Profile viewable by organizations (students only)
- Privacy controls: Optional fields for PII protection
- XSS prevention: Sanitize bio and text inputs
- URL validation: Prevent malicious link injection
- File upload security: Virus scanning, size limits, type restrictions
- Audit trail: Profile changes logged (US-15)

Alignment With Backend Design

This UI maps directly to:

Endpoint:

- ➢ GET /users/{user_id}/profile

- PATCH /users/{user_id}/profile Auto-Generated Fields:

Auto-Generated Fields:

Experience (from US-12):

- Populated when project marked complete
- Title: Derived from project role or type
- Organization: Project owner organization
- Duration: Start date to completion date
- Rating: From feedback (US-13)
- Cannot be manually edited

Achievements (auto-awarded):

- Top Performer: 3+ projects with 4.8+ rating
- Quick Learner: First project completed in ≤ 2 weeks
- Reliable Partner: 5+ projects completed
- Rising Star: First project with 5.0 rating
- Versatile: Projects in 3+ different skill categories
- Early Adopter: One of first 100 users

Profile Completeness Calculation:

- Profile picture: 15%
- Name: 10%
- Headline: 10%
- Bio (50+ chars): 15%
- Location: 5%
- Skills (5+): 20%
- Portfolio (1+ item): 15%
- Experience (1+ project): 10%
- Total: 100%

Smart Matching Integration (US-20):

- Profile Quality Score: 15% of matching algorithm
- Higher completeness = higher match scores
- Skills array used for skill matching (40% weight)

- Ratings from experience used for success history (20% weight)
- Portfolio demonstrates commitment (quality signal)

Public Profile Features:

- Shareable URL: /profiles/{user_id}
- Public view for organizations during application review
- Privacy controls for optional field visibility
- Professional presentation for networking

4.5 Smart Matching Algorithm (US-20 / SA-1)

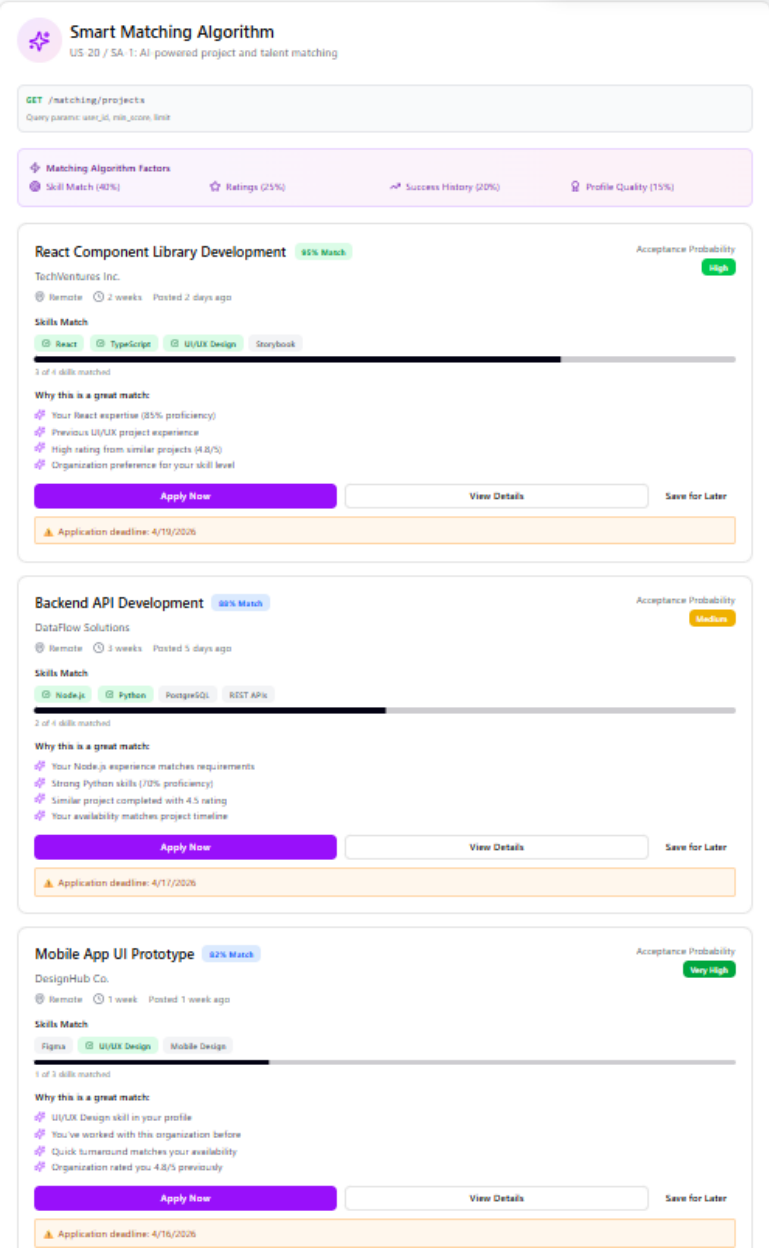


Figure 5 shows the wireframe for US-20 (Smart Matching Algorithm).

Description and Design Rationale

The Smart Matching Algorithm page provides AI-powered recommendations for projects (students) or candidates (organizations) based on comprehensive matching criteria.

This interface leverages machine learning and multi-factor analysis to present highly relevant matches, reducing search time, improving application acceptance rates, and

increasing project success rates. The algorithm considers skill alignment, historical performance, profile quality, ratings, availability, and behavioral patterns to generate match scores and detailed reasoning.

Layout Structure

The interface is structured into role-specific recommendation views:

Common Header Section:

1. Sparkles icon (gradient purple-pink theme)
2. Page title with SA-1 designation
3. User story reference
4. API endpoint documentation

Algorithm Overview Panel:

- Matching factors with weight percentages:
    - Skill Match: 40%
    - Ratings: 25%
    - Success History: 20%
    - Profile Quality: 15%
- Visual icons for each factor
- Purple-pink gradient background

Student View - Recommended Projects:

Each project match card includes:

1. Header Section
    a. Project title (large, prominent)
    b. Match score badge (color-coded: 90%+ green, 80%+ blue, 70%+ yellow)
    c. Organization name
    d. Location, duration, posted date with icons
    e. Acceptance probability badge (Very High, High, Medium, Low)
2. Skills Match Section
    a. Visual badges for all required skills
    b. Green checkmark for matched skills
    c. Gray for skills not in student profile
    d. Progress bar showing percentage match
    e. "X of Y skills matched" text
3. Match Reasons List

    a. Sparkles icon per reason
    b. AI-generated explanations for why it's a good match
    c. References to student's skills, experience, ratings, availability
4. Action Buttons
    a. Primary: "Apply Now" (purple, prominent)
    b. Secondary: "View Details" (outline)
    c. Tertiary: "Save for Later" (ghost)
5. Deadline Warning (conditional)
    a. Orange background for deadlines < 3 days
    b. Application deadline date display
    c. Warning icon

Organization View - Recommended Students:

Each student match card includes:

1. Header Section
    a. Student name (large, prominent)
    b. Match score badge (color-coded)
    c. Star rating display (aggregate from feedback)
    d. Location with icon
    e. Projects completed count with icon
    f. Response rate percentage
    g. Profile completeness badge
2. Skills Match Section
    a. Visual badges for all required skills (from project)
    b. Green checkmark for student's matching skills
    c. Gray for missing skills
    d. Progress bar showing skill coverage
    e. "X of Y required skills" text
3. Match Reasons List
    a. AI-generated explanations
    b. References skill alignment, ratings, experience, availability
    c. Behavioral signals (response rate, completion rate)
4. Availability Indicator
    a. Green background panel
    b. Availability status (Immediate, Within 1 week, etc.)
    c. Prominent display for timeline matching
5. Action Buttons

    a. Primary: "Invite to Apply" (purple)
    b. Secondary: "View Profile" (outline)
    c. Tertiary: "Save Candidate" (ghost)

Matching Methodology Panel:

- Detailed breakdown of algorithm factors
- Weighting explanations
- Behavioral considerations
- Preference matching

Continuous Improvement Panel:

- Machine learning approach
- Feedback loop integration (US-13)
- A/B testing notice
- Model retraining frequency

Benefits Panel:

- Split view: Students vs. Organizations
- Value propositions for each role
- Outcome improvements (higher acceptance, less screening, etc.)

Usability Considerations

Match Score Visualization:

- Color-coded badges for quick assessment
- 90%+ green (excellent match)
- 80-89% blue (good match)
- 70-79% yellow (fair match)
- Below 70% gray (poor match, not typically shown)

Skills Transparency:

- Visual representation of skill overlap
- Green badges for matched skills immediately recognizable
- Progress bar reinforces percentage match
- Clear "X of Y" textual summary

Explainable AI:

- "Why this is a great match" section for every recommendation
- Specific, personalized reasons
- References to user's profile data
- Builds trust in algorithm

Actionable Recommendations:

- Primary CTA prominently styled (purple)
- Multiple action options (apply, view, save)
- Clear next steps for each match
- No dead-ends in user journey

Urgency Indicators:

- Deadline warnings for time-sensitive opportunities
- Orange highlight for <3 days remaining
- Application deadline clearly displayed
- Encourages timely action

Profile Quality Signals:

- Profile completeness percentage (organizations viewing students)
- Rating stars (both views)
- Response rate (organization view)
- Projects completed count
- Builds confidence in recommendations

Card-Based Layout:

- Clean separation between matches
- Scannable format for quick comparison
- Hover effects for interactivity
- Responsive grid for various screen sizes
- Validation & Error

Validation & Error Handling

Match Generation Validation:

- Minimum match score threshold (typically 70%)

- At least 1 skill match required
- Active projects/available students only
- No duplicates in results

Empty State Handling:

- "No matches found" message
- Suggestions to improve profile (US-19)
- Recommendation to adjust preferences
- Link to browse all projects/students

Data Freshness:

- Real-time score calculation
- Profile changes reflected immediately
- Project status updates impact results
- Cache invalidation on key changes

Access Control:

- Role-based view (students see projects, orgs see students)
- JWT authentication required
- No cross-role data leakage
- Privacy-preserving recommendations

Algorithm Errors:

- Graceful fallback to basic matching
- Error logging for ML failures
- Default to recency if scoring fails
- Notification to admins for persistent issues

Security Alignment

- Accessible to: All authenticated users (role-based results)
- JWT authentication required
- Privacy protection: Only public profile data used
- No bias enforcement: Fair matching across demographics
- Audit trail: Match views and actions logged (US-15)
- Rate limiting: Prevent algorithm gaming

- Data minimization: Only necessary fields in response
- Anonymized training data: ML models trained on aggregated, anonymized data

Alignment With Backend Design

This UI maps directly to:

Endpoints:

- GET /matching/projects (for students)
- GET /matching/students (for organizations)

Matching Algorithm Components:

1. Skill Match (40% weight):
    a. Skill Score = (Matched Skills / Required Skills) × Proficiency Factor × Recency Factor
    b. Exact skill name matching
    c. Proficiency level from profile (US-19)
    d. Recency based on last used in project
    e. Weighted by skill importance to project
2. Ratings (25% weight):
    a. Rating Score = (User Average Rating / 5.0) × Project Count Factor
    b. Aggregate rating from feedback (US-13)
    c. More projects = more reliable rating
    d. Recent ratings weighted higher
    e. Minimum 1 rating required for factor
3. Success History (20% weight):
    a. Success Score = Completion Rate × On-Time Rate × Quality Factor
    b. Project completion percentage
    c. On-time delivery percentage
    d. Deliverable approval rate (US-11)
    e. Domain-specific success (similar projects)
4. Profile Quality (15% weight):
    a. Profile Score = Completeness % × Verification Factor × Activity Factor
    b. Profile completeness from US-19
    c. Verified email/phone (future)
    d. Recent platform activity
    e. Portfolio quality signals
5. Additional Factors:
    a. Availability Match: Student available when project starts

    b. Location Preference: Remote vs. local preference
    c. Duration Fit: Student prefers projects of this length
    d. Domain Interest: Student has indicated interest in this domain
    e. Previous Collaboration: Worked together before (bonus points)
    f. Response Time: Historical application response speed
    g. Acceptance Likelihood: Probability student will accept if offered
6. Match Reason Generation:
    a. Natural language explanation per factor
    b. Personalized to user's profile data
    c. References specific skills, projects, ratings
    d. Adaptive templates with data injection
7. Continuous Learning:
    a. Outcome Tracking: Application acceptance/rejection (US-8)
    b. Project Success: Completion and ratings (US-12, US-13)
    c. Feedback Loop: High match scores → successful projects = reinforce
    d. Feature Importance: Adjust weights based on outcomes A/B
    e. Testing: Test algorithm variants for improvement
    f. Model Retraining: Weekly retraining on new data
8. Bias Mitigation:
    a. No gender, age, or demographic data in model
    b. Fair representation across all user segments
    c. Regular bias audits on match distributions
    d. Diversity in recommendations
    e. Compliance with fair hiring practices
9. Integration Points:
    a. Profile data (US-19): Skills, bio, portfolio, completeness
    b. Ratings (US-13): Average rating, feedback quality
    c. Completions (US-12): Success rate, on-time delivery
    d. Deliverables (US-10, US-11): Quality and approval rate
    e. Applications (US-8): Historical acceptance rates
    f. Analytics (US-18): Performance metrics for scoring
10. Acceptance Probability Estimation:
    a. Historical acceptance rate for similar match scores
    b. Organization's typical acceptance criteria
    c. Student's application success rate
    d. Competition level for the project
    e. Time sensitivity and urgency

5. Definition of Done (Sprint 4)

The Definition of Done was refined for Sprint 4 and applied consistently to all backlog items.

A User Story is considered complete when:

Design

- Data model is designed and reviewed
- Database schema aligns with requirements
- API contract is clearly defined
- Endpoint request/response structures documented

Implementation

- Database migration successfully applied
- Backend endpoint implemented and fully functional
- Business logic correctly enforces requirements
- Role-Based Access Control (RBAC) enforced
- Input validation implemented (edge cases handled)
- Frontend components implemented (if applicable)
- Frontend successfully integrated with backend APIs

Testing

- Unit tests written for core logic
- Edge case scenarios covered
- API endpoints tested (success + failure cases)
- Manual functional testing completed
- No failing test cases

Quality & Integration

- Code coverage ≥ 90% fornew core business logic
- Code reviewed and approved
- Pull request merged into main branch
- No regression introduced to previous features
- System builds and runs without errors

Documentation

- API endpoints documented
- Code comments added where necessary
- User flow documented (if applicable)

Deployment Readiness

- Feature is deployable without breaking the existing system
- Environment variables/configurations updated as needed

6. Acceptance Criteria

6.1 US-20 / SA-1 – Smart Matching Algorithm

Requirement: SA-1 | User Story: US-20 | Story Points: 18

Acceptance Criteria:

- Every open project receives a computed match score in the range [0, 100] when requested by an authenticated student.
- The five weighted components (Skill Match 40%, Experience Match 20%, Interest Match 15%, Activity Score 10%, Success Score 15%) each contribute correctly to the final weighted formula.
- Rankings are sorted in descending order — the highest-scoring project receives rank = 1.
- Results are reproducible: identical inputs always produce identical scores and rank order.
- Projects with no skill overlap with the student's profile return a low score (not an error).
- New students with no history receive neutral scores (50) for Success and Experience factors —they are not penalised for having no record.
- Projects with empty required_skills return a neutral Skill Match score (50) rather than zero.
- Only open projects are included in recommendations — closed and completed projects are excluded.
- Organization users are denied access to the recommendations endpoint (HTTP 403).
- Students without a profile receive HTTP 404 with a meaningful message.

- An empty list is returned gracefully when no open projects exist — not an error.
- The optional top_n query parameter (1–100) correctly slices results to the N best matches.
- Test coverage for the scoring engine is ≥ 90%.

Returns:

Scenario	HTTP Status	Response Body
Student with profile + open projects	200 OK	[{ "project_id": 12, "title": "...", "match_score": 87.5, "rank": 1, "skill_score": 100.0, ... }]
Student with no profile	200 OK	{ "detail": "Student profile not found..." }
No open projects exist	200 OK	[] (empty array)
Organization user	403 Forbidden	{ "detail": "Access forbidden" }
Unauthenticated request	401 Unauthorized	{ "detail": "Not authenticated" }
Returns top 5 ranked results	200 OK	Array of max 5 RecommendationItem objects
Invalid param (must be ≥ 1)	422 Unprocessable Entity	Validation error detail

7. Source Code Development Progress

7.1 Repository

GitHub Repository: https://github.com/chazdj/MicroMatch

Project board integrated with GitHub Issues for traceability between user stories and commits.

7.2 Summary of Contributions

Backend Contributions

- Implemented matching_engine.py as a pure service layer with no SQLAlchemy dependencies, using three DTOs (StudentDTO, ProjectDTO, ScoredProject) to decouple the engine from ORM models.
- Built five factorized scoring helpers: compute_skill_match() (set intersection), compute_experience_match() (tiered + duration penalty), compute_interest_match() (major + Jaccard bio blend), compute_activity_score()

(30-day engagement tiers), and compute_success_score() (completion, delivery, and rating blend).

- Implemented score_student_project() applying the weighted formula and rank_projects() for O(N log N) sorted ranking with top-N slicing. All functions are divide-by-zero safe.
- Created RecommendationItem Pydantic schema and GET /recommendations endpoint with _build_student_dto() (six DB queries) and _build_project_dto() helper functions.
- Applied require_role('student') RBAC, 404 guard for missing profiles, open-only project filtering, graceful empty list handling, and optional top_n query parameter (1–100). Router registered in main.py.

Frontend Contributions

- Created frontend/src/components/RecommendationWidget.jsx: self-contained component that fetches from GET /recommendations, renders ranked project cards, and handles all state.
- Score badge color-coded by match quality: green ≥70%, yellow ≥40%, red <40%.
- Score breakdown bar (primary color progress bar) and per-card skill/experience/interest percentage display.
- Quick Apply CTA button calls POST /applications; shows per-card success/error inline without page reload.
- Handles no-profile state (404) with a prompt to complete profile — not an error state.
- Handles empty recommendations and network errors gracefully.
- Updated frontend/src/pages/Home.jsx: students see RecommendationWidget; organizations and admins see role-appropriate quick-link cards.
- Responsive grid: 1 column mobile, 2 column tablet, 3 column desktop.

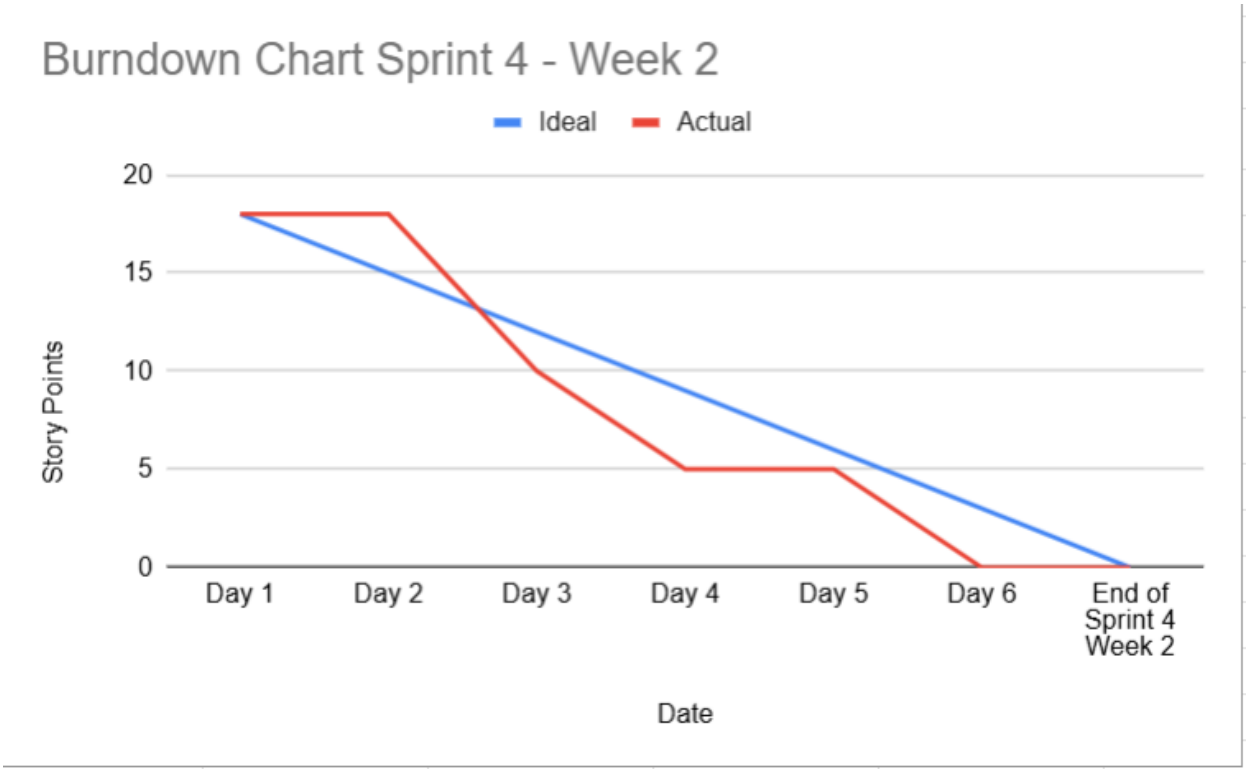
Integration Work

- Verified end-to-end flow: student profile + open projects → DTO assembly → engine scoring → API response → UI render.
- Confirmed no regression to existing endpoints via full pytest suite.
- Validated RBAC: organization token correctly returns 403 on /recommendations.
- Confirmed top_n=0 rejected at API layer (422) while engine-level behavior documented in tests.

7.3 Important Commits

Commit	Related Issue	User Story
feat: matching score engine	#89	US-20
feat: recommendation api	#90	US-20
test: recommendation api	#89/90	US-20
ui: recommendation added to home dashboard	#91	US-20

7.4 Burndown Chart



Sprint Capacity: 18 Story Points
Sprint Duration: 6 Development Days

Burndown Chart Analysis

The burndown chart reflects a back-loaded but fully delivered sprint. Days 1–2 were spent building the engine foundation with no story points burned, as the entire 18 SP block was

released together once Issue #89 reached completion on Day 3. This is characteristic of algorithm-heavy work where all components must integrate before any issue can be closed. Days 3–6 then show rapid burn as the API and UI layers built directly on top of the validated engine

Velocity Alignment:

Sprint 1 average velocity: ~16 points
Sprint 2 average velocity: ~14 points
Sprint 3 average velocity: ~16 points
Sprint 4 Week 2 completed: 18 points

8. Software Testing

8.1 Requirement-to-Test Mapping

Functional Requirement	User Story	Test Case ID
SA-1	US-20	TC-US20-01 – TC-US20-09

8.2 User Acceptance Test Specification

All test cases are mapped to SA-1, ensuring full coverage of sprint scope.

TC-US20-01 – Ranked Recommendations Returned for Student with Profile

Related Requirement: SA-1
User Story: US-20 – Smart Matching Algorithm
Actor: Student

Preconditions:

- Student account exists in the system with role = 'student'.
- Student has a completed profile with at least one skill (e.g., 'python, fastapi, sql'), a declared major (e.g., 'Computer Science'), and a bio.
- At least two open projects exist with differing required_skills — one with high overlap with the student's skills and one with no overlap.
- The student is not currently authenticated; no active session exists.

Steps:

1. Navigate to the application login page at /login.
2. Enter the student's registered email address in the email field.
3. Enter the student's correct password in the password field.
4. Click the 'Login' button and verify successful redirection to the student dashboard.
5. Open a REST client or use the browser's developer console network tab.
6. Construct a GET request to http://localhost:8000/recommendations.
7. Add the Authorization header with value 'Bearer {token}' using the JWT token returned at login.
8. Send the GET request and observe the HTTP status code returned.
9. Inspect the response body and confirm it is a JSON array.
10. For each item in the array, verify the presence of: project_id, title, match_score, rank, skill_score, experience_score, interest_score, activity_score, and success_score.
11. Record the match_score and rank values for each item in the response.
12. Verify that items are listed in descending match_score order — the first item should have the highest score and rank = 1.
13. Confirm that the project with the highest skill overlap with the student's profile appears at rank 1.

Expected Result:

- HTTP 200 OK is returned.
- Response is a non-empty JSON array of RecommendationItem objects.
- Each item contains all nine fields: project_id, title, match_score, rank, skill_score, experience_score, interest_score, activity_score, success_score.
- All individual scores are in the range [0.0, 100.0].
- Items are sorted by match_score descending — index 0 has the highest score and rank = 1.
- The project with the best skill overlap with the student's profile ranks highest.

Postconditions:

- Student can view personalised ranked project recommendations on the dashboard.
- Rankings are consistent with the algorithm's weighted formula.
- The RecommendationWidget on the dashboard displays the same ranked results.

TC-US20-02 – No Profile Returns 404

Related Requirement: SA-1
User Story: US-20 – Smart Matching Algorithm
Actor: Student

Preconditions:

- Student account exists in the system with role = 'student'.
- The student has NOT completed profile creation — no StudentProfile record exists for this user.
- At least one open project exists in the system.
- The student is not currently authenticated.

Steps:

1. Navigate to the application login page at /login.
2. Enter the student's registered email address and correct password.
3. Click 'Login' and confirm successful authentication — JWT token is issued.
4. Open a REST client or browser developer tools.
5. Construct a GET request to http://localhost:8000/recommendations.
6. Add the Authorization header with the JWT token obtained at login.
7. Send the GET request.
8. Observe the HTTP status code in the response.
9. Inspect the response body's 'detail' field for a descriptive error message.

Expected Result:

- HTTP 404 Not Found is returned.
- The response detail field contains the word 'profile' and indicates the student must create one.
- No recommendation data or project list is included in the response body.

Postconditions:

- Student is informed they need a profile to receive recommendations.
- No partial or incorrect data is returned.
- The system remains in a consistent state — no records are created or modified.

TC-US20-03 – Closed and Completed Projects Excluded

Related Requirement: SA-1
User Story: US-20 – Smart Matching Algorithm
Actor: Student

Preconditions:

- Student account exists with role = 'student' and has a completed profile.
- Three projects have been created by an organization: Project A with status = 'open', Project B with status = 'closed', Project C with status = 'completed'.
- All three projects have required_skills populated.
- The student is not currently authenticated.

Steps:

1. Navigate to /login and authenticate as the student using valid credentials.
2. Verify the JWT token is successfully returned and stored.
3. Construct a GET request to http://localhost:8000/recommendations.
4. Add the Authorization header with the student's JWT token.
5. Send the request and observe the HTTP status code.
6. Inspect the response body and extract the list of project_id values returned.
7. Record the project_id of Project A (open), Project B (closed), and Project C (completed).
8. Verify which project IDs appear in the recommendations response.
9. Confirm that Project B's project_id and Project C's project_id do not appear anywhere in the response.
10. Confirm that Project A's project_id is present in the response.

Expected Result:

- HTTP 200 OK returned.
- Only Project A (status = 'open') appears in the response.
- Project B (closed) and Project C (completed) are absent from the result set.
- No error is raised for the absence of inactive projects.

Postconditions:

- Only open, active projects are surfaced to students.
- Closed and completed projects remain in the database unmodified.

TC-US20-04 – Organization User Denied Access

Related Requirement: SA-1
User Story: US-20 – Smart Matching Algorithm
Actor: Organization

Preconditions:

- An organization account exists in the system with role = 'organization'.
- The organization is not currently authenticated.
- At least one open project and one student profile exist in the system.

Steps:

1. Navigate to /login.
2. Enter the organization's registered email address and correct password.
3. Click 'Login' and confirm that authentication succeeds — a JWT token is issued with role = 'organization'.
4. Open a REST client or browser developer tools network panel.
5. Construct a GET request to http://localhost:8000/recommendations.
6. Add the Authorization header using the organization's JWT token.
7. Send the request.
8. Observe the HTTP status code returned.
9. Inspect the response body for any project or recommendation data.

Expected Result:

- HTTP 403 Forbidden is returned.
- Response body contains a detail message indicating access is forbidden.
- No project, score, or recommendation data is included in the response.

Postconditions:

- RBAC correctly restricts recommendations endpoint to students only.
- Organization's token is not usable to access student-facing features.
- No data leakage occurs.

TC-US20-05 – Unauthenticated Request Denied

Related Requirement: SA-1
User Story: US-20 – Smart Matching Algorithm
Actor: Un

Preconditions:

- No active user session exists.
- No valid JWT token is available.
- The application server is running and accessible.

Steps:

1. Open a REST client or browser developer tools.
2. Construct a GET request to http://localhost:8000/recommendations.
3. Do NOT add an Authorization header — send the request with no authentication credentials.
4. Send the request.
5. Observe the HTTP status code in the response.
6. Inspect the response body for any recommendation data or token-related error message.
7. Optionally, repeat the request with a malformed Authorization header (e.g., 'Bearer
8. invalid_token') and confirm the same 401 response is returned.

Expected Result:

- HTTP 401 Unauthorized returned.
- Response body contains a detail message indicating the user is not authenticated.
- No recommendation data, project list, or score information is returned.

Postconditions:

- Endpoint is fully protected — no data is accessible without a valid JWT.
- The system remains in a consistent state; no records are created or modified.

TC-US20-06 – Empty Projects List Returns Empty Array

Related Requirement: SA-1
User Story: US-20 – Smart Matching Algorithm
Actor: Student

Preconditions:

- Student account exists with role = 'student' and has a completed profile.
- No projects with status = 'open' exist in the system — all projects are either closed, completed, or the database has no projects at all.
- The student is not currently authenticated.

Steps:

1. Navigate to /login and authenticate as the student using valid credentials.
2. Confirm the JWT token is issued successfully.
3. Construct a GET request to http://localhost:8000/recommendations.
4. Add the Authorization header with the student's JWT token.
5. Send the request.
6. Observe the HTTP status code.
7. Inspect the response body and verify its structure.
8. Confirm that no error status or exception message is returned.

Expected Result:

- HTTP 200 OK returned — the endpoint does not treat zero open projects as an error.
- Response body is an empty JSON array: [].
- No error message, stack trace, or 4xx/5xx status code is returned.

Postconditions:

- System handles zero-project state gracefully without raising exceptions.
- Student's dashboard RecommendationWidget displays the appropriate empty state message.
- No records are modified.

TC-US20-07 – Scores Are Reproducible (Determinism)

Related Requirement: SA-1
User Story: US-20 – Smart Matching Algorithm
Actor: Student

Preconditions:

- Student account exists with role = 'student' and has a profile with at least one skill populated.
- At least three open projects exist with varying required_skills.
- No changes are made to the student profile, project data, or application history between the two requests.
- The student is not currently authenticated.

Steps:

1. Navigate to /login and authenticate as the student.
2. Obtain the JWT token from the login response.
3. Send a GET request to http://localhost:8000/recommendations with the Authorization header.
4. Record the full response: capture every project_id, match_score, and rank value from Response 1.
5. Without changing any data or logging out, immediately send an identical GET request to http://localhost:8000/recommendations again.
6. Record the full response: capture every project_id, match_score, and rank value from Response 2.
7. Compare Response 1 and Response 2 item by item.
8. Verify that the order of project_ids is identical between both responses.
9. Verify that every match_score value in Response 1 exactly matches the corresponding value in Response 2.
10. Verify that all rank assignments are identical between both responses.

Expected Result:

- Both responses return an identical list of project_ids in the same order.
- All match_score values are numerically identical between both calls.
- All rank assignments are identical between both calls.
- No non-deterministic ordering or score variance is observed.

Postconditions:

- Algorithm is confirmed deterministic — identical inputs always produce identical outputs.
- Students receive a consistent, stable ranking across page loads.

TC-US20-08 – top_n Query Parameter Slices Results

Related Requirement: SA-1
User Story: US-20 – Smart Matching Algorithm
Actor: Student

Preconditions:

- Student account exists with role = 'student' and has a completed profile with skills populated.
- At least five open projects exist in the system with varying required_skills.
- The student is not currently authenticated.

Steps:

1. Navigate to /login and authenticate as the student using valid credentials.
2. Obtain the JWT token from the login response.
3. First, send a GET request to http://localhost:8000/recommendations (no top_n parameter) with the Authorization header.
4. Record the full ranked list returned — note the project_id, match_score, and rank of all items. This is the baseline.
5. Now send a second GET request to http://localhost:8000/recommendations?top_n=3 with the same Authorization header.
6. Observe the HTTP status code of the second response.
7. Count the number of items returned in the second response.
8. Compare the project_ids of the top 3 items from the baseline (step 4) against the items returned by the top_n=3 response.
9. Verify the items in the sliced response are still sorted by match_score descending.
10. Send a third request with top_n=0 and observe the response code.

Expected Result:

- HTTP 200 OK returned for top_n=3 request.

- Exactly 3 items are returned in the top_n=3 response.
- The 3 returned items match the first 3 results from the unsliced baseline response — same project_ids and scores.
- Items in the sliced response remain sorted by match_score descending.
- HTTP 422 Unprocessable Entity returned for top_n=0 — the parameter is rejected as invalid.

Postconditions:

- top_n correctly limits results to the N highest-ranked projects.
- Invalid top_n values are rejected at the API layer before reaching the engine.

TC-US20-09 – Zero Skill Match Returns Low Score, Not Error

Related Requirement: SA-1
User Story: US-20 – Smart Matching Algorithm
Actor: Student

Preconditions:

- Student account exists with role = 'student'.
- Student profile has skills = 'python, fastapi' and major = 'Computer Science'.
- Exactly one open project exists with required_skills = 'verilog, fpga, embedded' — no overlap with the student's skills.
- The student has no completed projects or feedback ratings (new student).
- The student is not currently authenticated.

Steps:

1. Navigate to /login and authenticate as the student using valid credentials.
2. Obtain the JWT token from the login response.
3. Construct a GET request to http://localhost:8000/recommendations.
4. Add the Authorization header with the student's JWT token.
5. Send the request and observe the HTTP status code.
6. Inspect the response body — locate the returned project item.
7. Record the value of the skill_score field for the returned project.
8. Record the value of the match_score field for the returned project.
9. Verify the match_score falls within [0.0, 100.0] and is not null or an error.

10. Verify the project is present in the response (zero-match projects are not filtered out).
11. Confirm no exception, error detail, or 5xx status is returned.

Expected Result:

- HTTP 200 OK returned — no server error occurs despite zero skill overlap.
- skill_score = 0.0, reflecting zero intersection between student skills and required skills.
- match_score is a low but valid float in the range [0.0, 100.0] — other factors contribute non-zero values.
- The project is still present in the response list; zero-match projects are not silently dropped.
- No null values, stack traces, or divide-by-zero errors appear in the response.

Postconditions:

- Algorithm handles complete skill mismatch gracefully without errors.
- The skill_score of 0.0 is correctly factored into the weighted formula.
- Students can still receive recommendations for projects outside their current skill set.

9. Backlog Grooming

No changes were made to the sprint backlog or product backlog during Sprint 4 Week 2. Sprint execution followed the planned structure defined in Week 1. All three issues (#89, #90, #91) were delivered as scoped with no scope creep, no deferrals, and no estimate revisions.

10. Risk Management

Risk	Likelihood	Impact	Mitigation
Engine scores not deterministic across environments	Low	High	Pure function design with no datetime.now() inside scoring; only graduation_year used (stable)
SQLite timestamp precision causes	Medium	Low	Order by (created_at DESC, id DESC) in

test ordering failures			notifications; engine tests use id-based assertions
Jaccard interest score lower than expected for short bios	Medium	Low	Tests assert relative ordering (relevant > irrelevant), not absolute thresholds
top_n=0 at engine level returns all results	Low	Low	API layer rejects with 422 via ge=1 param; engine behavior documented in tests
Week 3 messaging introduces DB schema changes	Low	Medium	Message model scoped to new table; no changes to existing models required

11. Sprint 4 Week 2 Outcome

Sprint 4 Week 2 successfully delivered the complete Smart Matching & Ranking Algorithm (SA-1 / US-20) across all three implementation layers — the scoring engine, the API endpoint, and the student dashboard UI.

Sprint 4 Week 2 delivered:

- matching_engine.py — a pure, reusable service with five factorized scoring functions, a weighted formula, and O(N log N) ranking.
- GET /recommendations — a student-only FastAPI endpoint with DTO assembly, RBAC enforcement, and top_n pagination support.
- RecommendationWidget — a responsive React component rendering ranked project cards with score badges, score breakdowns, and per-card quick apply CTAs.
- 69 new tests (54 unit + 15 endpoint) with ≥ 90% coverage on all new scoring logic.

The sprint goal was fully achieved. MicroMatch now provides students with intelligent, personalized project recommendations ranked by a deterministic weighted algorithm. The platform is now positioned to begin Week 3 — Messaging, Analytics, and Profile Enhancements — which completes Sprint 4.